Creating Core Resources and Building the Diagnostic Ladder

Tim Warner

Principal Author, Pluralsight

TechTrainerTim.com

Four framing questions

How does the Deployment > ReplicaSet > Pod hierarchy enable self-healing workloads?

How do ClusterIP, NodePort, and LoadBalancer Services expose workloads differently?

How do Namespaces, labels, and annotations organize resources at scale?

How does the get > describe > logs > events diagnostic ladder solve every troubleshooting scenario?

"The product catalog API needs to be live by Thursday. Get it running, expose it to the QA team, and make sure you know how to fix it when it goes down. Because it will go down. And troubleshooting is 30% of your CKA score -- so consider this practice for both the job and the exam."

Priya S., Job Role, Globomantics Robotics Company

How does the controller hierarchy enable self-healing workloads?

Bare Pods vs. managed Deployments

Bare Pods

- Created directly with kubectl run or YAML
- No controller monitors or replaces failed Pods
- Deleted Pods are gone permanently
- No scaling or self-healing behavior
- Used for debugging, not production workloads

Managed Deployments

- Deployment manages ReplicaSets → Pods
- ReplicaSet continuously enforces desired replica count
- Failed or deleted Pods are automatically recreated
- Supports scaling, rolling updates, and rollbacks
- Standard production pattern for running workloads

The ReplicaSet reconciliation loop

ReplicaSet continuously compares desired count to actual count

If a Pod crashes or is deleted, a replacement is created automatically

If a node fails, Pods are rescheduled to healthy nodes

`kubectl scale deployment` adjusts the desired replica count

You almost never create bare Pods in production or on the exam

How do Services expose workloads to the network?

Services and the selector-label contract

A Service provides a stable virtual IP for a set of Pods

The Service selects Pods by matching their labels

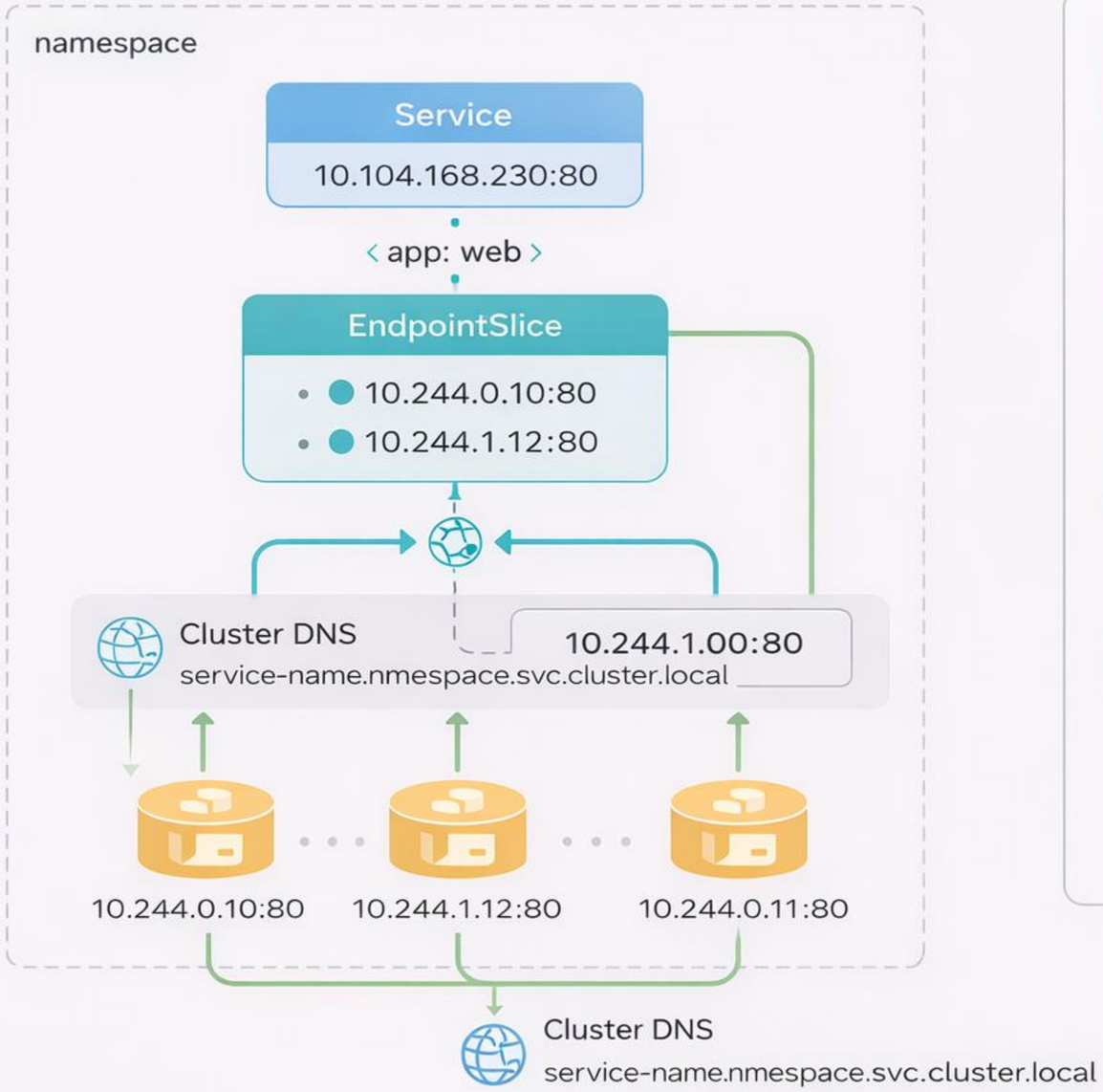
Pod IPs appear in EndpointSlices when labels match

If a Pod loses its label or fails readiness, its IP is removed

EndpointSlices are the modern replacement for the legacy Endpoints object

If labels match and readiness passes, Pod IPs are included in EndpointSlices

K8S request path: DNS ➔ Service ➔ Pod



get Current state

describe Pod events

logs Container output

events Cluster timeline

Follow this ladder to diagnose every troubleshooting scenario.

Three Service types

ClusterIP (default)

Internal-only virtual IP for service-to-service traffic

Stable endpoint for Pods selected by labels

Used by other Pods inside the cluster

DNS: `service-name.namespace.svc.cluster.local`

NodePort

Extends ClusterIP with a port on every node (30000–32767)

Accessible from outside the cluster via node IP + port

Routes traffic to the Service → then to Pods

Works in local clusters (kind, minikube)

LoadBalancer

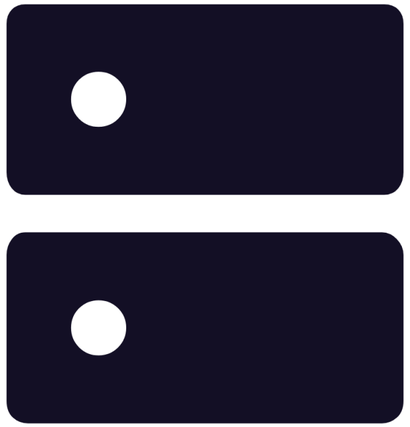
Extends NodePort with a cloud provider load balancer

Provides external IP for production access

Automatically routes traffic to healthy Pods

On kind: stays Pending (no cloud provider)

Verifying service connectivity with DNS



Every Service gets a DNS record: `service-name.namespace.svc.cluster.local`

Same-namespace Pods can use the short name: just ‘my-service’

Short names work via DNS search domains

Cross-namespace access requires the full DNS name

Run from inside a Pod (not your host):

```
kubectl run debug --image=busybox --rm
–it --nslookup my-service
```

```
kubectl run debug --image=busybox --rm
–it -- wget -qO- my-service
```

How do Namespaces, labels, and annotations organize resources?

Namespaces, labels, and annotations

Namespaces scope resource names and enable RBAC policies

Labels are for selection -- Services, ReplicaSets, and NetworkPolicies use them

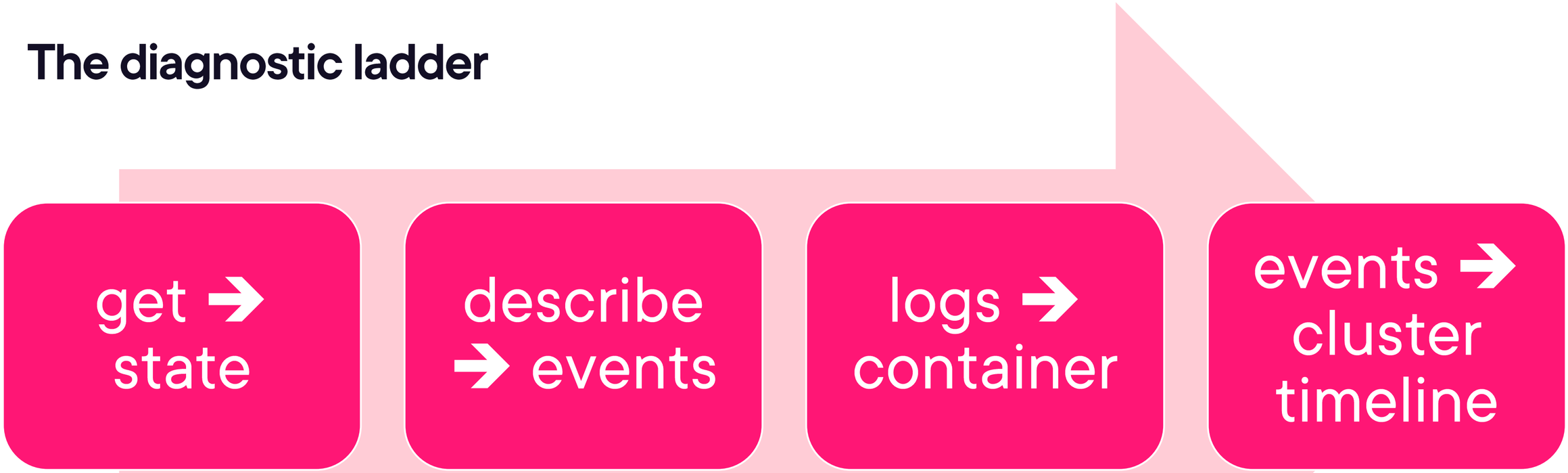
Annotations carry metadata but do not affect scheduling or routing

kubectl get pods -A queries across all namespaces

Compound selectors: kubectl get pods -l env=staging,team=backend

How does the diagnostic ladder solve every troubleshooting scenario?

The diagnostic ladder



Current status (Running, Pending, CrashLoopBackOff)

Scheduling, image pulls, probe failures

stdout / stderr from the container

Chronological, cross-resource events

Container output streams and multi-container Pods

`kubectl logs` reads stdout and stderr captured by containerd

For multi-container Pods: `kubectl logs pod-name -c cont-name`

`kubectl logs --previous` shows output from the last terminated container

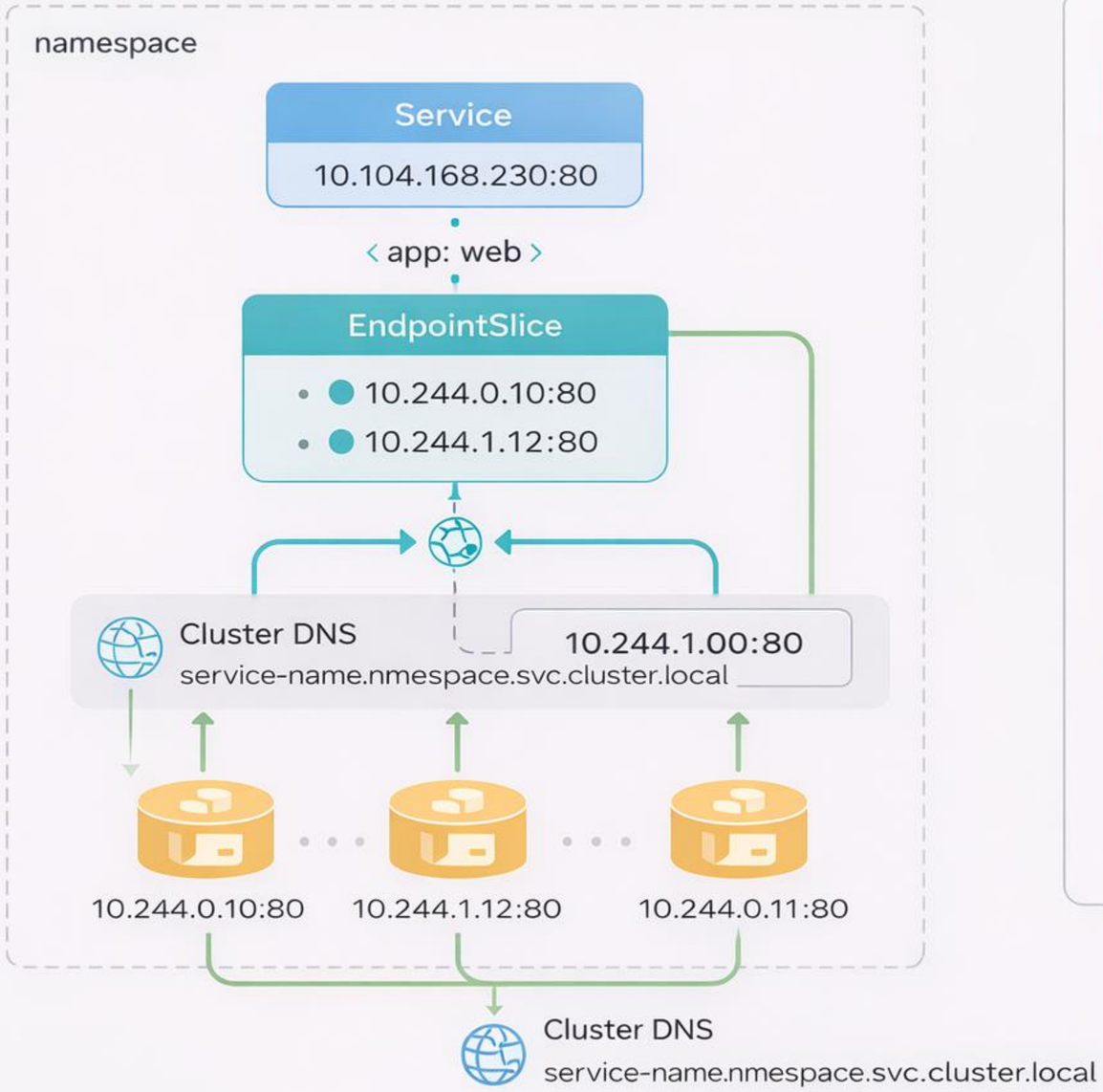
kubectl logs --follow streams output in real time

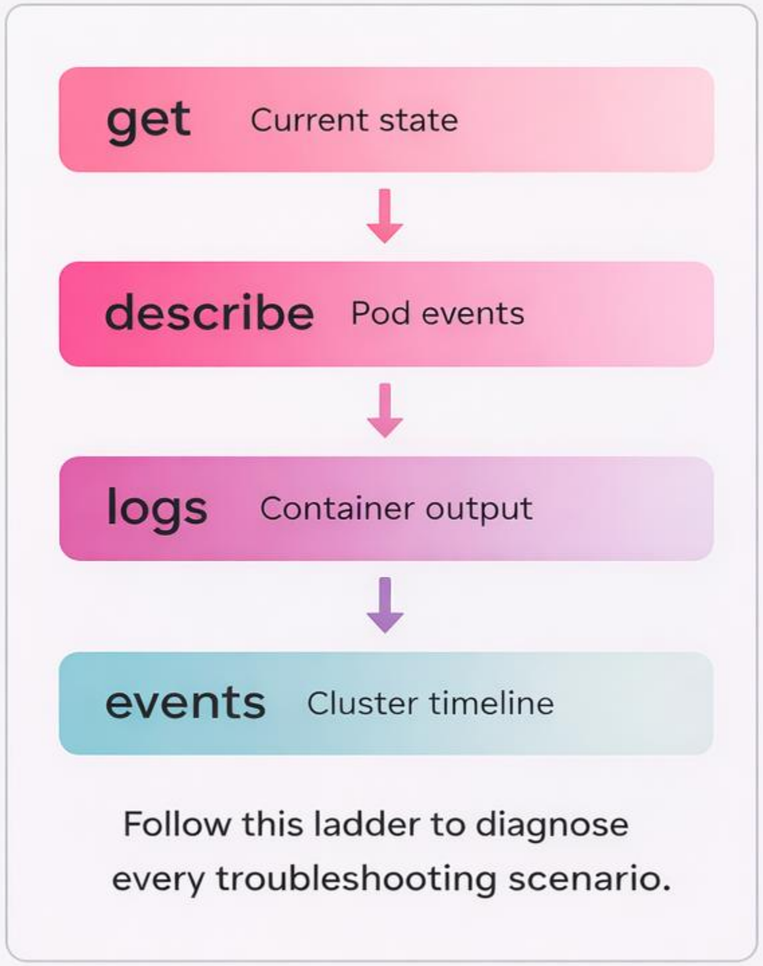
Container output management is a dedicated CKA competency

kubectl logs and multi-container pods

```
# Get logs from a single-container Pod
kubectl logs my-pod

# Multi-container Pod: MUST specify container
kubectl logs my-pod -c app-container

# Previous container instance (CrashLoopBackOff)
kubectl logs my-pod --previous

# Stream logs in real time (live debugging)
kubectl logs -f my-pod

# List containers in a Pod (before choosing -c)
kubectl describe pod my-pod
```

Diagnostic ladder applied to the request path





Demo

Deploy, expose, break, and diagnose

Checking out with Globomantics

“The catalog API went down Friday night. I ran get, describe, logs, events -- found the bad image tag in under two minutes. Three weeks ago that would have been a two-hour panic. The diagnostic ladder works.”

Priya S., Job Role, Globomantics Robotics Company

From Globomantics to you

Deploy with Deployments, not bare Pods

Self-healing comes from the ReplicaSet reconciliation loop -- bare Pods do not recover

Verify connectivity end-to-end

DNS resolution plus EndpointSlice inspection proves the full path from client to Pod

Organize with labels, not just namespaces

Labels drive selection for Services, ReplicaSets, and NetworkPolicies -- they are the wiring

Make the diagnostic ladder automatic

get, describe, logs, events -- this pattern is your weapon for 30% of the CKA score

Pluralsight

Tim Warner | TechTrainerTim.com